Lab 2 : Data Pipeline avec dbt & DuckDB

Data Engineering

Binôme :

Nabil Ramdani
Kenza El Maimouni

Encadré par :

Prof. Lamia BenHiba

École Centrale Casablanca
Février 2026

Table des matières

1 Architecture Finale du Pipeline **2**
1.1 Vue d'ensemble . . . 2
1.2 Composants de l'architecture . . . 2
1.3 Structure des couches . . . 3
1.4 Modélisation dimensionnelle (Kimball) . . . 3
1.4.1 Bus Matrix . . . 3

2 Implémentation du Pipeline **4**
2.1 Couche Staging . . . 4
2.2 Tests de Qualité des Données . . . 5
2.3 Couche Marts — Dimensions . . . 5
2.4 Couche Marts — Fait . . . 6

3 Chargement Incrémental (Incremental Loading) **7**
3.1 Problème adressé . . . 7
3.2 Solution dbt : Modèles Incrémentaux . . . 7

4 Slowly Changing Dimensions Type 2 (SCD2) **8**
4.1 Concept . . . 8
4.2 Implémentation avec dbt Snapshots . . . 8
4.3 Dimension historisée dim_apps_scd . . . 8
4.4 Test du SCD2 — Simulation d'un changement . . . 9

5 Serving Layer — Metabase **10**
5.1 Connexion à Metabase . . . 10
5.2 Visualisation 1 — Note Moyenne par Application . . . 10
5.3 Visualisation 2 — Évolution du Nombre de Reviews dans le Temps . . . 11

6 Comparaison : Pipeline Python vs Pipeline dbt **12**

7 Réflexions **13**
7.1 Partie la plus fragile de l'architecture . . . 13
7.2 Plus grand insight architectural . . . 13
7.3 Décision architecturale à changer . . . 13

8 Lien GitHub **14**

9 Conclusion **15**

Architecture Finale du Pipeline

Vue d'ensemble

Le pipeline implémenté dans ce lab suit une architecture de type **Data Warehouse** basée sur trois couches distinctes : ingestion, transformation, et serving. L'objectif est de remplacer la logique Python ad-hoc du Lab 1 par des frameworks standards de data engineering.

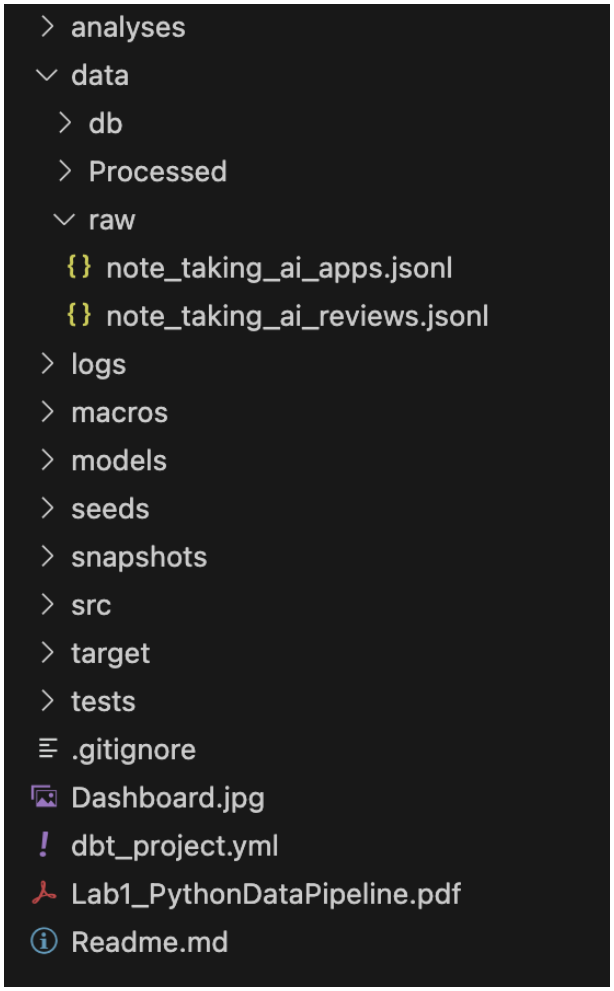

FIGURE 1 – Structure du projet dbt

Composants de l'architecture

L'architecture repose sur trois technologies complémentaires :

— **Python (Lab 1)** : Responsable de l'ingestion des données via l'API Google Play Store. Il produit des fichiers JSONL bruts (`note_taking_ai_apps.jsonl` et `note_taking_ai_reviews.jsonl`) stockés dans `data/raw/`.

— **dbt Core** : Orchestre toute la logique de transformation via des modèles SQL versionnés, testés et documentés. Il structure les données en couches staging et marts.

— **DuckDB** : Sert à la fois de moteur d'exécution analytique et de couche de stockage. Il matérialise les vues staging et les tables marts dans un fichier `playstore.duckdb`.

Structure des couches

Couche	Modèles	Matérialisation
Staging	stg_playstore_apps, stg_playstore_reviews	View
Marts (Dimensions)	dim_apps, dim_categories, dim_developers, dim_date	Table
Marts (Faits)	fact_reviews	Incremental Table
Snapshots	dim_apps_snapshot	SCD Type 2

Table 1 – Structure des couches du pipeline

Modélisation dimensionnelle (Kimball)

Nous avons adopté la méthodologie Kimball pour concevoir notre star schema :

— **Processus métier** : L'évaluation (review) d'une application mobile sur le Google Play Store.

— **Grain** : Une ligne dans `fact_reviews` représente *un avis laissé par un utilisateur sur une application à un instant donné.*

— **Dimensions** : dim_apps (quelle app ?), dim_categories (quelle catégorie ?), dim_developers (quel développeur ?), dim_date (quand ?)

— **Faits mesurables** : rating (moyenne), thumbs_up_count (somme), nombre de reviews (count)

1.4.1 Bus Matrix

Faits / Dimensions	dim_apps	dim_categories	dim_developers	dim_date
fact_reviews	X	X	X	X

Table 2 – Bus Matrix

Implémentation du Pipeline

Couche Staging

La couche staging constitue l'interface contrôlée entre les données brutes et les transformations métier. Elle effectue uniquement :

- Le renommage des colonnes (ex : `appId` → `app_id`)
- Le cast des types (ex : `score` → `numeric`, `at` → `timestamp`)
- La création de clés de substitution (surrogate keys) via MD5
- La suppression des enregistrements avec des clés nulles

Listing 1 – Extrait de stg_playstore_apps.sql

```sql
with raw_apps as (
    select *
    from read_json_auto('/path/to/note_taking_ai_apps.jsonl')
)
select
    md5(cast(appId as varchar))     as app_key,
    appId                           as app_id,
    title                           as app_name,
    cast(score as numeric)          as catalog_rating,
    genre                           as category_name
from raw_apps
where appId is not null
```

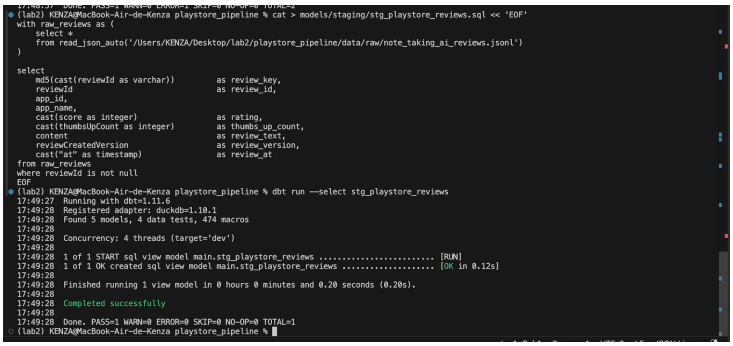

FIGURE 2 – Exécution réussie des modèles staging

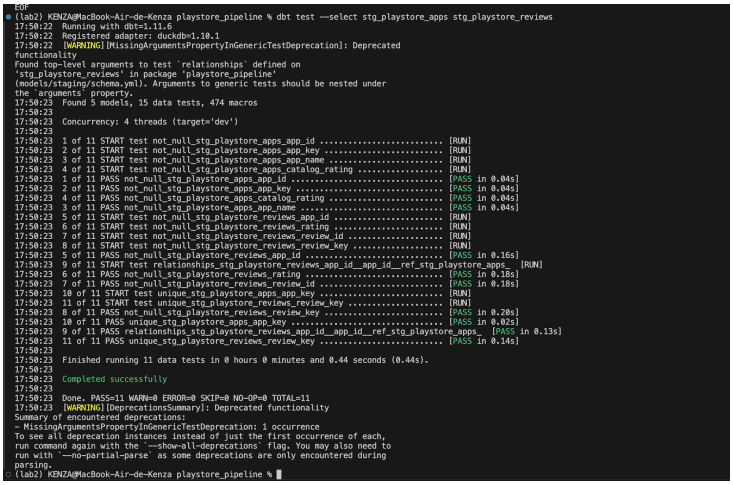

FIGURE 3 – Exécution réussie des modèles staging

Tests de Qualité des Données

Nous avons implémenté des tests génériques dbt dans le fichier `schema.yml` :

— **not_null** : sur toutes les clés primaires et champs critiques
— **unique** : sur les surrogate keys (app_key, review_key)
— **relationships** : intégrité référentielle entre reviews et apps

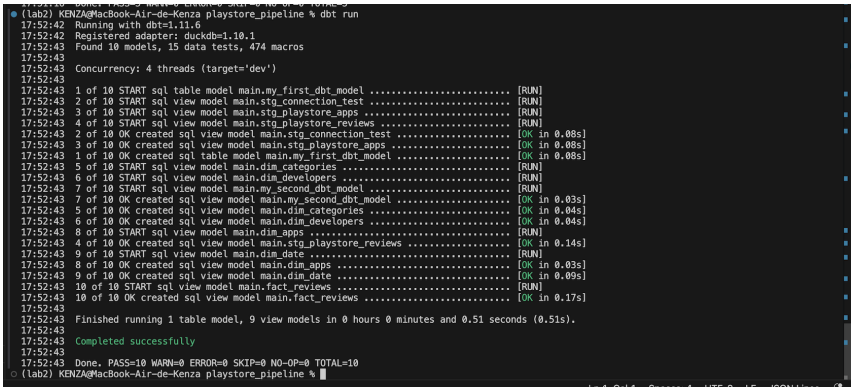

FIGURE 4 – Résultats des tests de qualité — 10/10 PASS

Couche Marts — Dimensions

Chaque dimension est construite à partir de la couche staging et possède une clé de substitution unique :

Listing 2 – Extrait de dim_categories.sql

```
with categories as (
    select distinct
        md5(category_id)  as category_key,
        category_id,
        category_name
    from {{ ref('stg_playstore_apps') }}
    where category_id is not null
```

```
)
select * from categories
```

La dimension `dim_date` génère une dimension temporelle continue basée sur les dates des reviews, avec des attributs comme `year`, `month`, `quarter`, `day_of_week` et `is_weekend`. La clé de date suit le format YYYYMMDD (ex : 20260204) conformément aux bonnes pratiques Kimball.

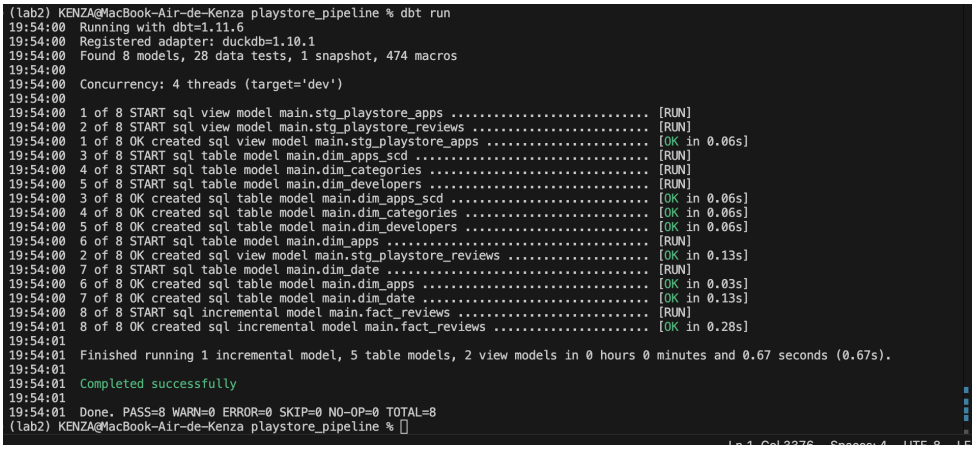

FIGURE 5 – Pipeline complet — staging en views, marts en tables

Couche Marts — Fait

La table `fact_reviews` centralise les mesures analytiques en jointure avec toutes les dimensions via leurs clés de substitution :

Listing 3 – Extrait de fact_reviews.sql

```
select
    r.review_key                                        as review_id,
    a.app_key,
    d.developer_key,
    cast(strftime(r.review_at, '%Y%m%d') as integer) as date_key,
    r.rating,
    r.thumbs_up_count,
    r.review_text,
    r.review_version
from {{ ref('stg_playstore_reviews') }} r
left join {{ ref('dim_apps') }} a on r.app_id = a.app_id
left join {{ ref('dim_developers') }} d on a.developer_key = d.
    developer_key
```

Chargement Incrémental (Incremental Loading)

Problème adressé

Sans chargement incrémental, notre pipeline reconstruit entièrement la table `fact_reviews` à chaque exécution. Cela pose trois problèmes : coût de calcul élevé, insertions dupliquées, et recomputation inutile.

Solution dbt : Modèles Incrémentaux

dbt résout ce problème via les **incremental models**. Lors du premier run, toutes les données sont chargées. Lors des runs suivants, seules les nouvelles données (filtrées par `is_incremental()`) sont insérées ou mises à jour.

Listing 4 – Configuration incrémentale de fact_reviews.sql

```
{{
    config(
        materialized='incremental',
        unique_key='review_id'
    )
}}

{% if is_incremental() %}
    and r.review_at > (select max(review_at)
                       from {{ ref('stg_playstore_reviews') }}
                       where review_key in
                             (select review_id from {{ this }}))
{% endif %}
```

La logique incrémentale fonctionne ainsi :

- `unique_key='review_id'` : dbt utilise cette clé pour éviter les doublons (upsert)
- `is_incremental()` : bloque de filtre actif uniquement lors des runs suivants
- Seules les reviews avec une date supérieure aux reviews existantes sont chargées

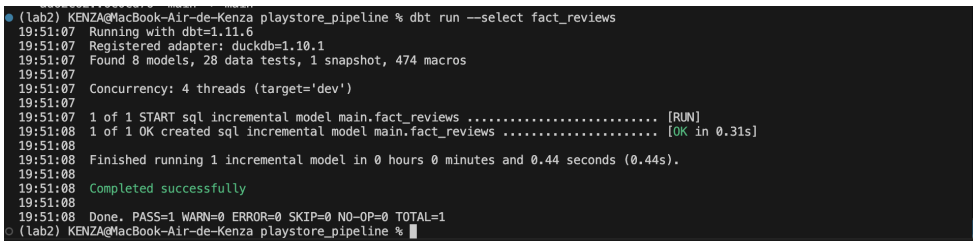

FIGURE 6 – Exécution du modèle incrémental fact_reviews

Slowly Changing Dimensions Type 2 (SCD2)

Concept

Les Slowly Changing Dimensions (SCD) traitent l'évolution des données dans le temps. Le **Type 2** conserve l'historique complet : chaque changement crée une nouvelle version de l'enregistrement, l'ancienne étant marquée comme expirée.

Implémentation avec dbt Snapshots

dbt implémente le SCD2 via les **snapshots**. Nous avons utilisé la stratégie **check** qui compare les colonnes définies à chaque run.

Listing 5 – dim_apps_snapshot.sql

```
{% snapshot dim_apps_snapshot %}
{{
    config(
        target_schema='main',
        unique_key='app_id',
        strategy='check',
        check_cols=['app_name', 'category_name', 'developer_name', 'price']
    )
}}
select app_key, app_id, app_name, developer_name,
       category_name, price, is_paid, catalog_rating
from {{ ref('stg_playstore_apps') }}
{% endsnapshot %}
```

dbt ajoute automatiquement les colonnes de métadonnées :

- `dbt_valid_from` : date de début de validité de la version
- `dbt_valid_to` : date de fin de validité (NULL = version courante)

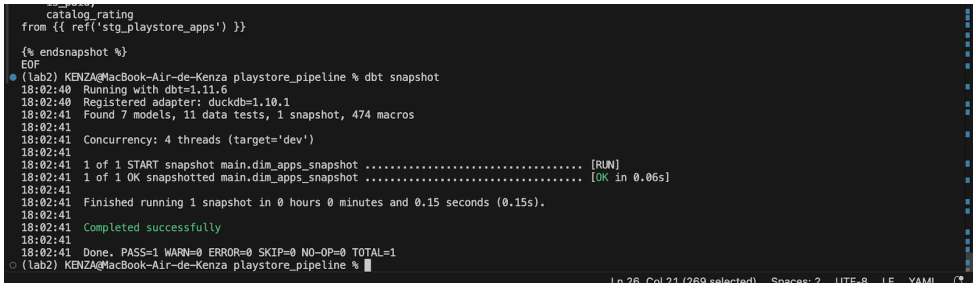

FIGURE 7 – Exécution du snapshot SCD2

Dimension historisée dim_apps_scd

La table `dim_apps_scd` transforme le snapshot en dimension propre avec une colonne `is_current` pour faciliter les requêtes sur les valeurs courantes :

Listing 6 – Extrait de dim_apps_scd.sql

```
select
    app_key, app_id, app_name, developer_name,
    category_name, price, catalog_rating,
    dbt_valid_from,
    dbt_valid_to,
    case when dbt_valid_to is null then true
         else false end as is_current
from {{ ref('dim_apps_snapshot') }}
```

Test du SCD2 — Simulation d'un changement

Pour valider le SCD2, nous avons simulé un changement de catégorie dans les données sources : l'app **Google NotebookLM** est passée de `Productivity` à `Education`.

Après avoir relancé `dbt snapshot`, nous avons vérifié que deux versions coexistent dans la table snapshot :

Listing 7 – Résultat de la requête de vérification SCD2

```
('Google NotebookLM', 'Productivity',
 datetime(2026-02-22 18:02:41), datetime(2026-02-22 18:21:48))
('Google NotebookLM', 'Education',
 datetime(2026-02-22 18:21:48), None)
```

La première ligne est l'ancienne version (expirée), la deuxième est la version courante (`dbt_valid_to = None`).

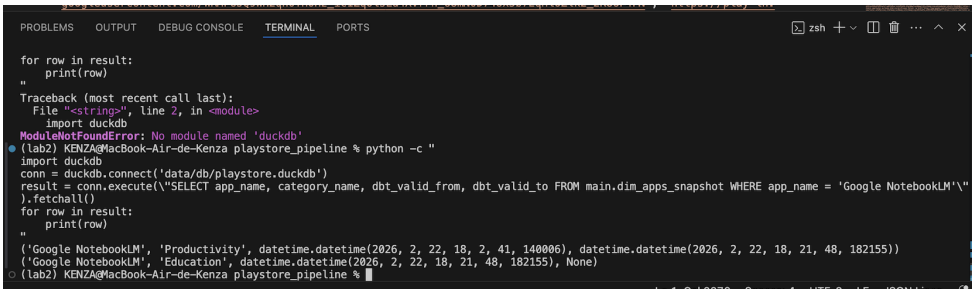

FIGURE 8 – Validation SCD2 — deux versions historiques de dim_apps

Serving Layer — Metabase

Connexion à Metabase

Pour démontrer la consommation analytique du star schema, nous avons connecté **Metabase** (outil BI open-source) à notre base DuckDB. Les données ont été exportées vers SQLite pour assurer la compatibilité avec Metabase v0.48.

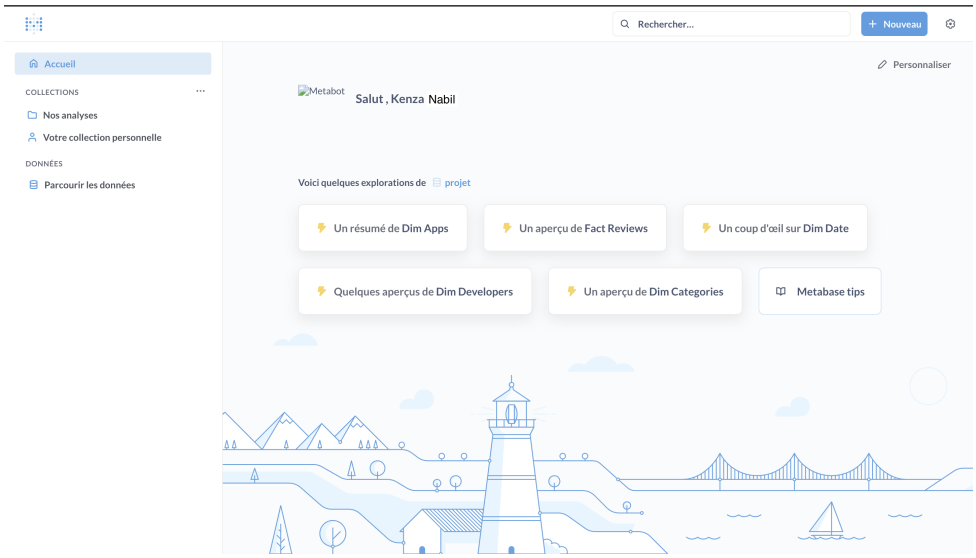

FIGURE 9 – Metabase connecté — toutes les tables du star schema visibles

Visualisation 1 — Note Moyenne par Application

FIGURE 10 – Note moyenne par application — certaines apps atteignent 4.5/5

Cette visualisation montre que la majorité des apps de prise de notes AI obtiennent des notes entre 3.5 et 4.5, avec une app qui se distingue négativement à 2.2.

Visualisation 2 — Évolution du Nombre de Reviews dans le Temps

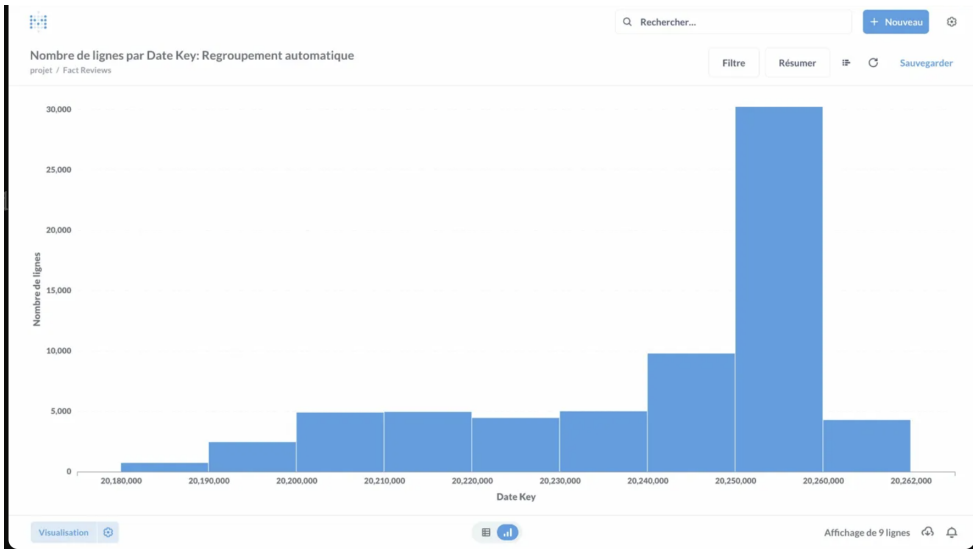

FIGURE 11 – Évolution temporelle des reviews — pic d'activité visible

Cette visualisation révèle une croissance significative du nombre de reviews dans le temps, avec un pic important sur la période récente, témoignant de l'adoption croissante des apps de prise de notes basées sur l'IA.

Comparaison : Pipeline Python vs Pipeline dbt

Critère	Pipeline Python (Lab 1)	Pipeline dbt (Lab 2)
Transformation	Logique Python ad-hoc, difficile à maintenir	SQL déclaratif versionné, réutilisable
Tests	Tests manuels à écrire from scratch	Tests génériques intégrés (not_null, unique, relationships)
Reproductibilité	Dépend de l'environnement Python	Entièrement reproductible via `dbt run`
Documentation	Commentaires manuels	Documentation auto-générée via schema.yml
Incrémental	Logique manuelle complexe	Natif via `materialized='incremental'`
SCD2	Code Python complexe	Natif via `dbt snapshot`
Dépendances	Gestion manuelle de l'ordre d'exécution	DAG automatique via `ref()`
Courbe d'apprentissage	Faible si Python connu	Moyenne (SQL + concepts dbt)

Table 3 – Comparaison Python vs dbt

Le pipeline dbt est clairement supérieur pour les pipelines analytiques complexes. La séparation ingestion/transformation/serving apportée par dbt améliore significativement la maintenabilité, la testabilité et la reproductibilité du pipeline.

Réflexions

Partie la plus fragile de l'architecture

La partie la plus fragile de notre architecture est la **couche d'ingestion** — les chemins absolus vers les fichiers JSONL dans les modèles staging. Si les fichiers sont déplacés ou renommés, tout le pipeline échoue silencieusement. Une amélioration serait d'utiliser des **sources dbt** (`sources.yml`) qui permettent de centraliser et versionner les références aux fichiers sources, avec des tests de fraîcheur (`freshness`).

Plus grand insight architectural

Le plus grand insight architectural est la **puissance de la séparation des responsabilités**. En forçant une distinction claire entre staging (nettoyage) et marts (business logic), dbt nous oblige à réfléchir à chaque transformation de manière isolée. Cela rend le débogage trivial : si une dimension est incorrecte, on sait exactement dans quel modèle chercher. Cette clarté est impossible à atteindre avec un pipeline Python monolithique.

Décision architecturale à changer

Si nous devions refaire ce projet, nous aurions utilisé les **sources dbt** dès le départ pour référencer les fichiers JSONL, plutôt que des chemins absolus dans le SQL. Nous aurions également créé une structure de dossiers plus granulaire dans marts (`marts/dimensions/` et `marts/facts/`) conformément aux recommandations du lab, ce qui aurait rendu la navigation dans le projet encore plus intuitive.

Lien GitHub

Le code source complet du projet est disponible sur le repository GitHub suivant :

`https://github.com/RamdaniNabil/python-data-engineering-lab`

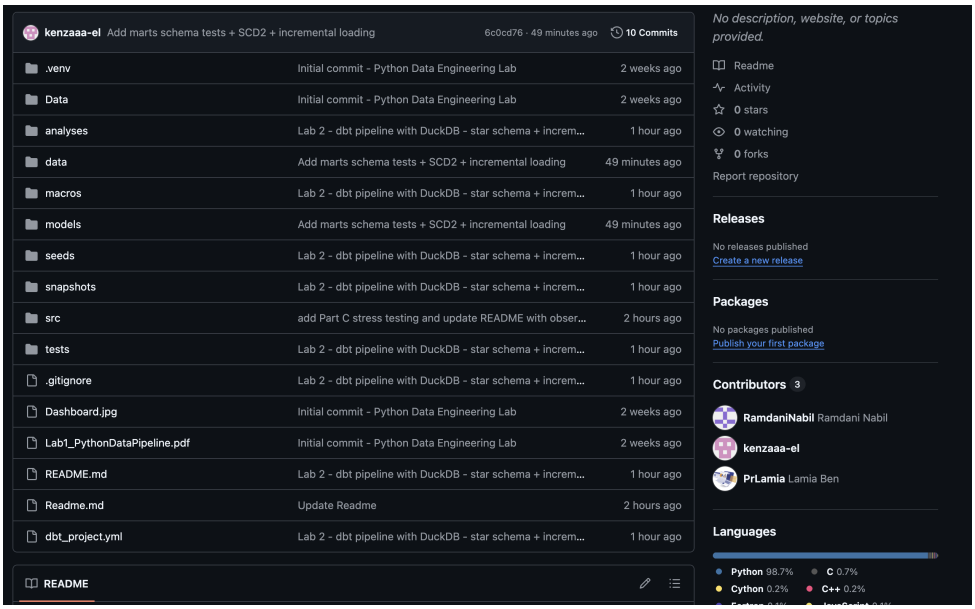

FIGURE 12 – Repository GitHub avec le code Lab 2

Conclusion

Ce lab nous a permis de transformer un pipeline Python ad-hoc en une architecture data engineering professionnelle et robuste. Grâce à dbt et DuckDB, nous avons implémenté :

— Un **star schema** complet suivant la méthodologie Kimball
— Une **couche staging** propre avec renommage, casting et surrogate keys
— Des **tests de qualité** automatisés (11/11 PASS)
— Un **chargement incrémental** pour `fact_reviews`
— Un **SCD Type 2** pour historiser les changements de `dim_apps`

La combinaison dbt + DuckDB s'est révélée particulièrement adaptée pour des pipelines analytiques locaux : DuckDB offre des performances analytiques remarquables sans infrastructure lourde, tandis que dbt apporte structure, testabilité et reproductibilité à la logique de transformation.